

Sunbeam Institute of Information Technology
Pune and Karad

Algorithms and Data structures

Trainer - Devendra Dhande

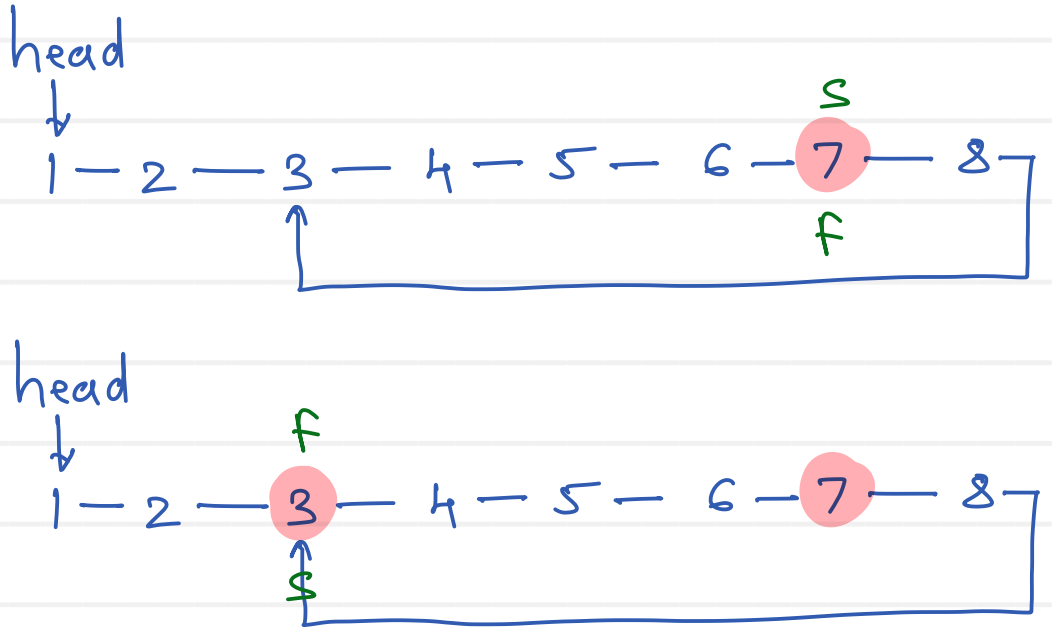
Email – devendra.dhande@sunbeaminfo.com

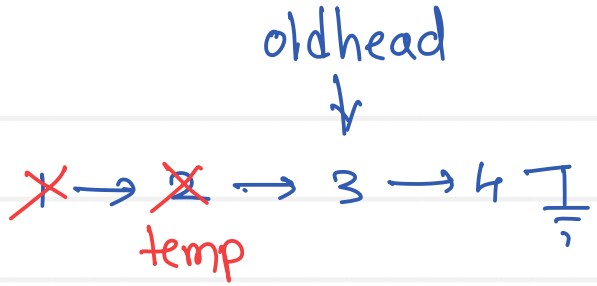
1. find middle — $n/2$
2. reverse second half — $n/2$
3. compare both halfs — $n/2$
4. reverse second half — $n/2$

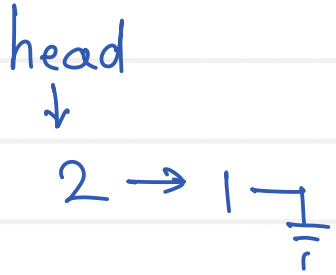
$4n$

$= 2n$

Floyd Cyclic Detection algorithm :

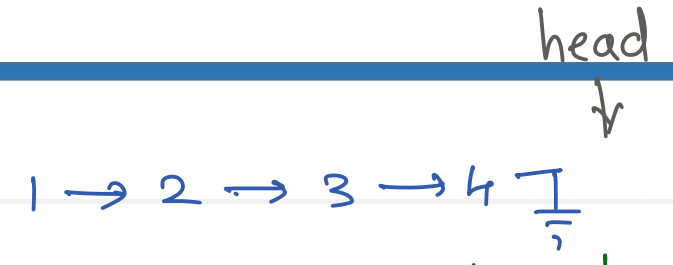


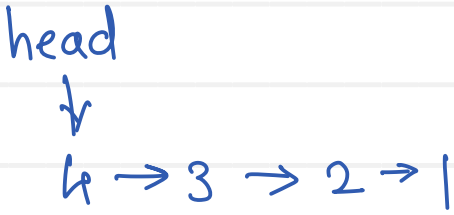





```
Node oldhead = head;
head = null
while(oldhead != null){
    Node temp = oldhead;
    oldhead = oldhead.next;
    temp.next = head
    head = temp
}
```

oldhead	head	temp
&f1	null	&f1
&f2	&f1	&f2
&f3	&f2	&f3
&f4	&f3	&f4
null	&f4	

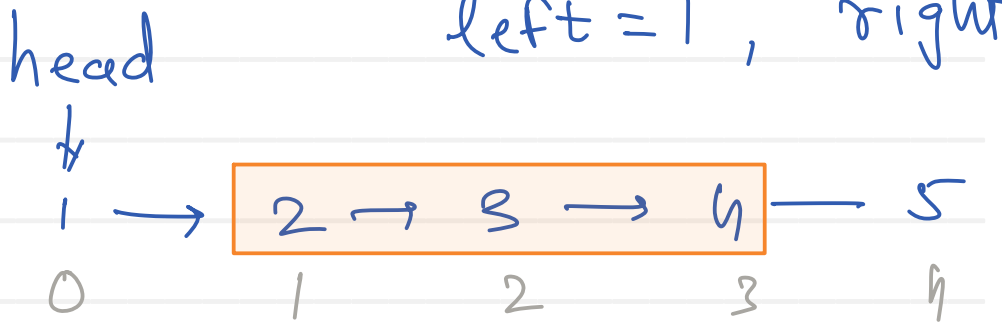


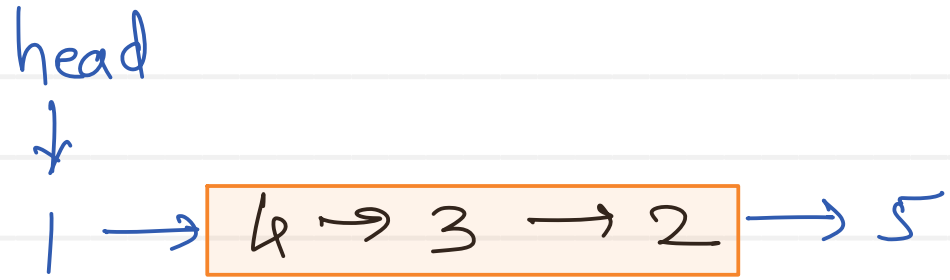



```
t1 = null;
t2 = head;
while (t2 != null) {
    head = t2.next;
    t2.next = t1
    t1 = t2;
    t2 = head;
}
head = t1;
```

t1	t2	head
null	&f1	&f2
&f1	&f2	&f3
&f2	&f3	&f4
&f3	&f4	null
&f4	null X	
		&f4

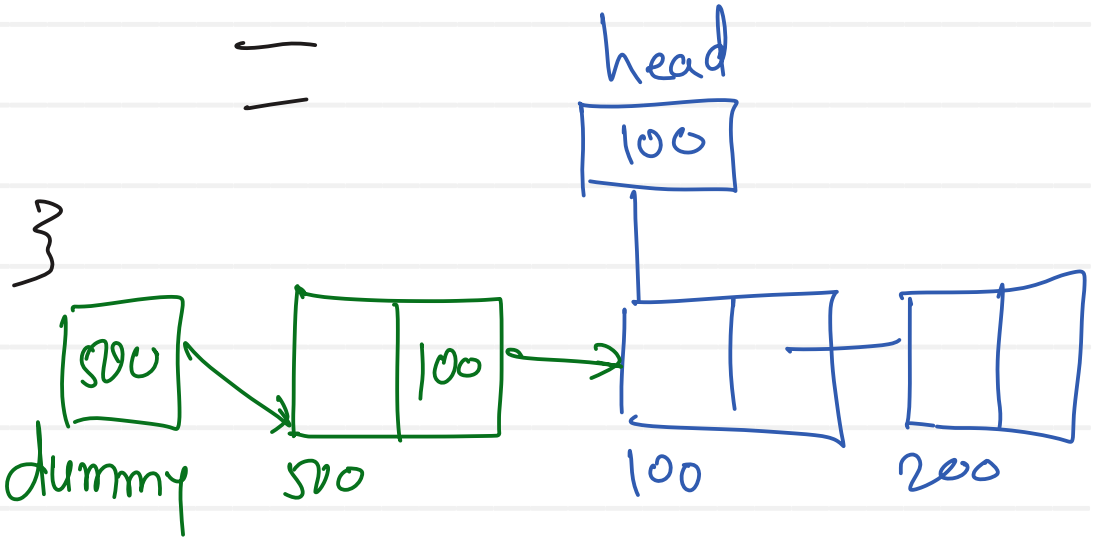
left = 1 , right = 3






```
reverseBetween(head, left, right){

    Node dummy = new Node(0);
    dummy.next = head;
```



Hashing

Array : linear search — O(n)
binary search — O(log n)

Linked List : search — O(n)

Hashing :
technique which allows to search data in constant time (O(1))

- implementation of hashing is called as hash table / hash map

- hashing is a technique in which data can be inserted, deleted and searched in constant average time O(1) (Associative data access)
- Implementation of hashing is known as hash table
- Hash table is array of fixed size in which elements are stored in key - value pairs

Array - Hash table
Index - Slot

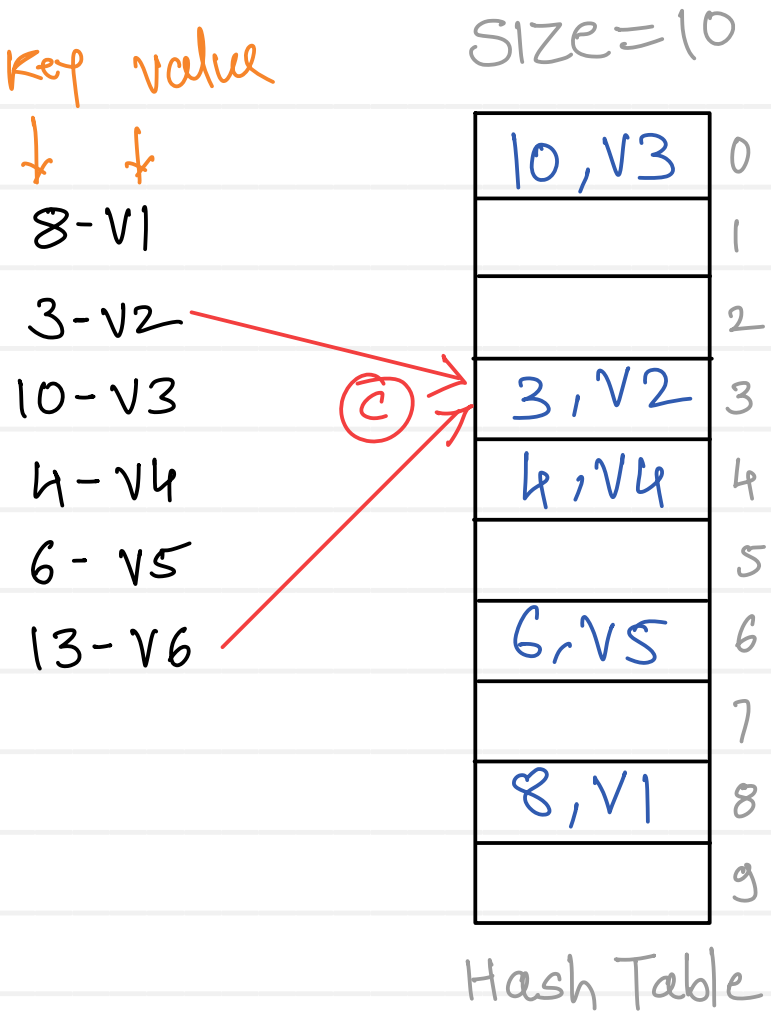
- In hash table only unique keys are stored
- Every key is mapped with one slot of the table and this is done with the help of mathematical function known as hash function



Hash Functions

1. Division method — $k \% m$
2. Multiplication method - $[m * (k * A)]$ $0 > A < 1$
3. Mid-square method — k^2 & find middle digit
4. Folding method — divide in two part, sum them, $\% m$
5. Cryptographic hashing — MD5, SHA-1 and SHA-256
6. Universal hashing - $((a.k+b) \% p) \% m$
7. Perfect hashing — collision free hash function for static set of key.
   - guarantees that no two keys will hash to same value

Hashing



h(k) = k % size ← hash function

h(8) = 8 % 10 = 8 ← hash code
h(3) = 3 % 10 = 3
h(10) = 10 % 10 = 0
h(4) = 4 % 10 = 4
h(6) = 6 % 10 = 6
h(13) = 13 % 10 = 3
c

Collision :
When we get same slot for multiple keys (distinct), this situation is called as collision

Add: – O(1)
1. slot = h(key)
2. arr[slot] = data

Search: – O(1)
1. slot = h(key)
2. return data.value

Delete : – O(1)
1. slot = h(key)
2. arr[slot] = null

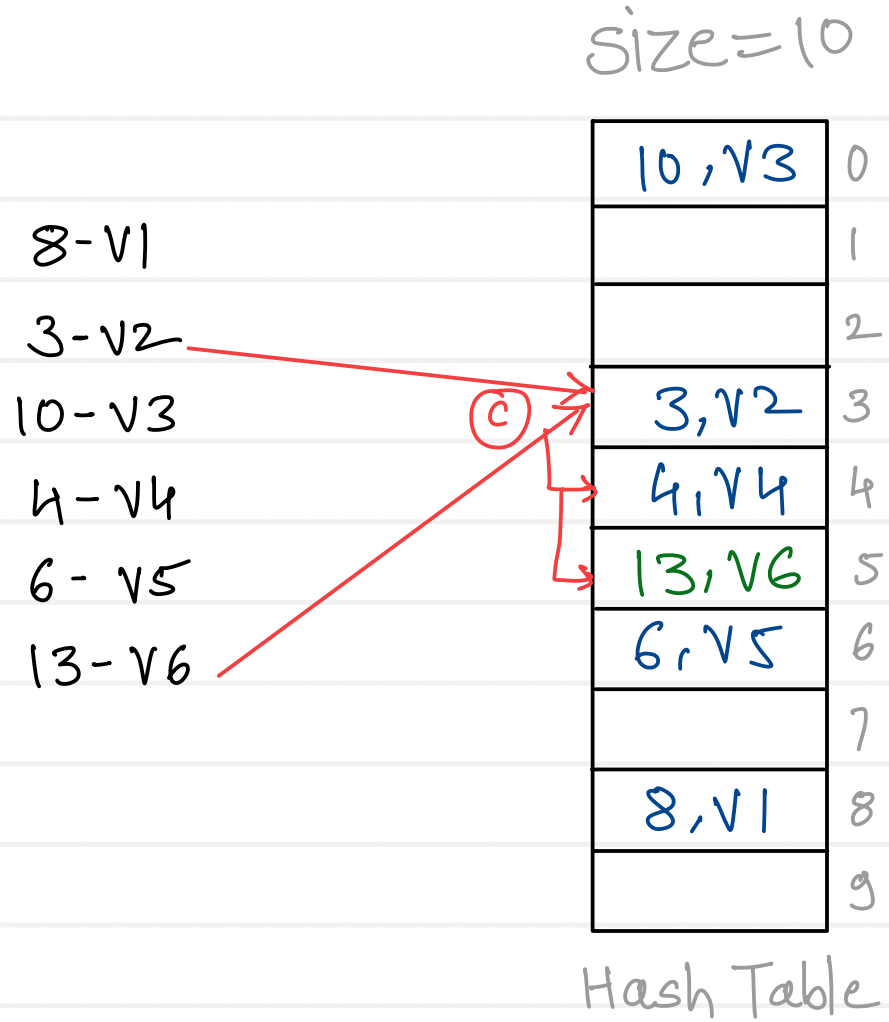
Collision handling / resolution techniques :

1. open addressing
   i. linear probing
   ii. quadratic probing
   iii. double hashing
2. Closed addressing / chainging

Probing :

Process of finding next empty slot when collision will occur

Open addressing - Linear probing



h(k) = k % size

h(k, i) = [h(k) + f(i)] % size

f(i) = i ← probe number

where i = 1, 2, 3, …

h(13) = 13 % 10 = 3 ©

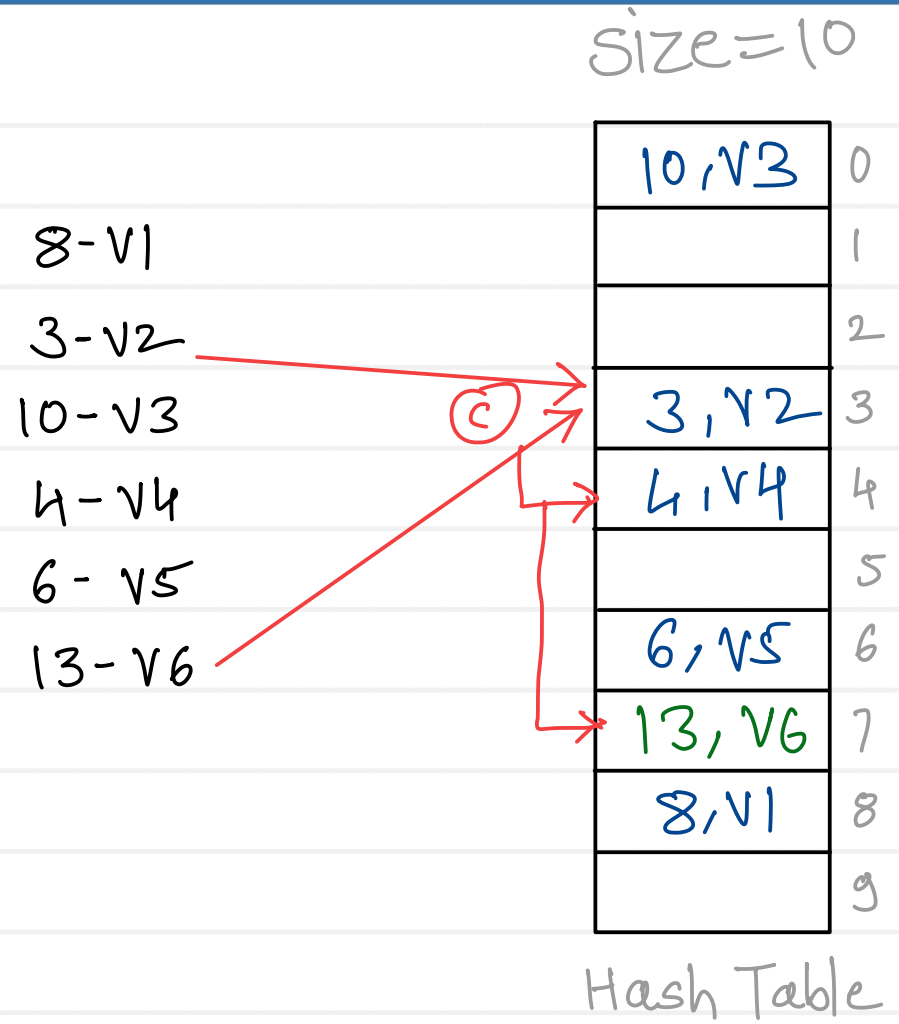
1st probe : h(13, 1) = [3 + 1] % 10 = 4 ©

2nd probe : h(13, 2) = [3 + 2] % 10 = 5

Primary clustering :

1. table becomes crowded / bulky near key position
2. needs to take long run of filled slots near key position to find empty slot

Open addressing - Quadratic probing



h(k) = k % size

h(k, i) = [h(k) + f(i)] % size

f(i) = i^2

where i = 1, 2, 3, …

h(13) = 13 % 10 = 3 (c)

1st probe : h(13,1) = [3+1] % 10 = 4 (c)

2nd probe : h(13,2) = [3+4] % 10 = 7

- primary clustering problem is solved

Secondary clustering :

needs to take long run of filled slots away key position to find empty slot

- there is no garantee of getting free slot for the key value pair

Open addressing - Quadratic probing

size=10

8-V1
3-V2
10-V3
4-V4
6-V5
13-V6
23,V7
33,V8

10,V3	0
	1
23,V7	2
3,V2	3
4,V4	4
	5
6,V5	6
13,V6	7
8,V1	8
33,V8	9

Hash Table

h(k) = k % size

h(k, i) = [h(k) + f(i)] % size

f(i) = i^2

where i = 1, 2, 3, …

h(23) = 23 % 10 = 3 ©
h(23,1) = [3+1] % 10 = 4 ©
h(23,2) = [3+4] % 10 = 7 ©
h(23,3) = [3+9] % 10 = 2

h(33) = 23 % 10 = 3 ©
h(33,1) = [3+1] % 10 = 4 ©
h(33,2) = [3+4] % 10 = 7 ©
h(33,3) = [3+9] % 10 = 2 ©
h(33,4) = [3+16] % 10 = 9

Open addressing - Double hashing

size = 11

8 - V1
3 - V2
10 - V3
25 - V4
36 - V5

	0
	1
	2
3, V2	3
	4
	5
25, V4	6
	7
8, V1	8
36, V5	9
10, V3	10

Hash Table

h1(k) = k % size
h2(k) = 7 - (k % 7)

h(k, i) = [h1(k) + i * h2(k)] % size

$h_1(8) = 8 \% 11 = 8$
$h_1(3) = 3 \% 11 = 3$
$h_1(10) = 10 \% 11 = 10$
$h_1(25) = 25 \% 11 = 3$ (C)
$h_2(25) = 7 - (25 \% 7) = 3$
$h(25, 1) = [3 + 1 * 3] \% 11 = 6$

$h_1(36) = 36 \% 11 = 3$ (C)
$h_2(36) = 7 - (36 \% 7) = 6$
$h(36, 1) = [3 + 1 * 6] \% 11 = 9$

Rehashing

$$\text{Load factor} = \frac{n}{N}$$

(λ)

$n = 7, N = 10$

$\lambda = \frac{7}{10} = 0.7$

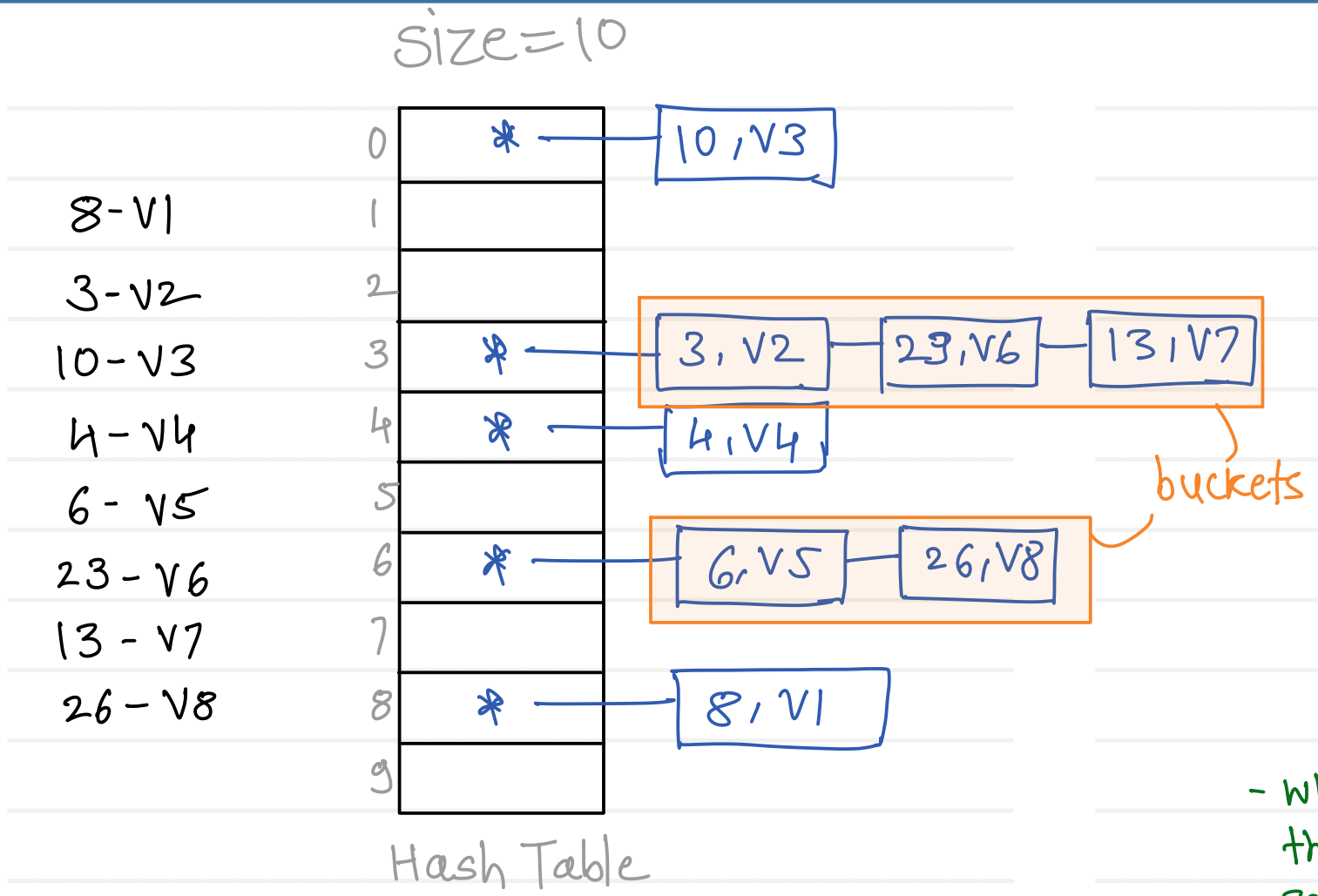
70 % filled

n - number of elements (key-value) present in hash table
N - number of total slots in hash table

- Load factor ranges from 0 to 1.
- If n < N Load factor < 1 - free slots are available
- If n = N Load factor = 1 - free slots are not available

- In rehashing, whenever hash table will be filled more than 60 or 70 % size of hash table is increased by twice
- Existing key value pairs are remapped according to new size

Closed Addressing / Chaining / Separate Chaining



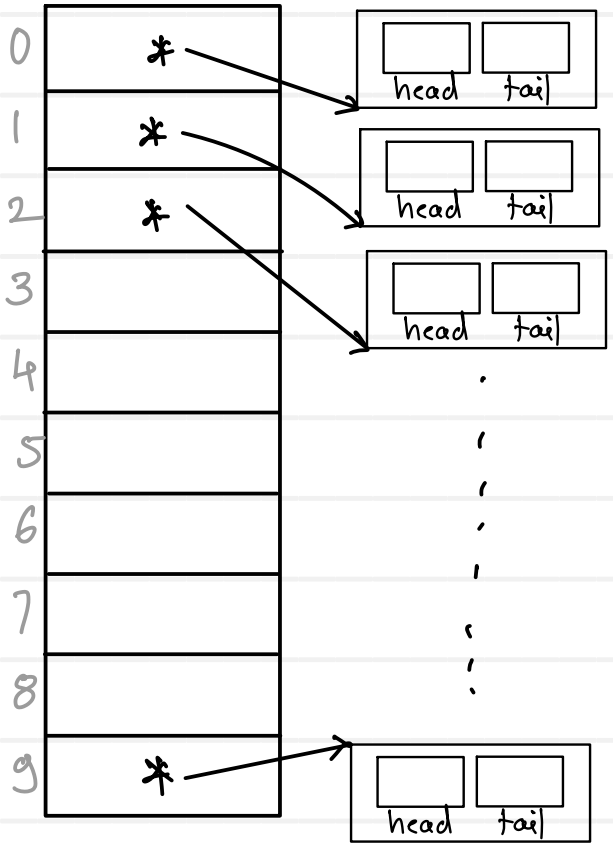
h(k) = k % size

- All buckets other than key position are closed for key-value pair, that why it is called as closed addressing.

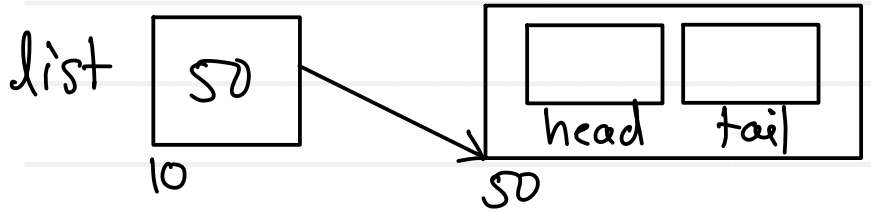
Advantage:
- no limit on key-value pairs

Disadvantage:
- space requirement is more
- key-value pairs are kept outside the table
- worst case time complexity O(n) – linked list

- When linked list of any slot crosses the threshold of length, it is converted into self balancing binary search trees. due which worst case time complexity will be O(log n)




```
LinkedList<Integer> list = new LinkedList<>();
```




```
LinkedList<Integer> arr[] = new LinkedList<>[SIZE];
```

or

```
List<Integer> arr[] = new List[SIZE];
for(i=0; i<SIZE; i++)
    arr[i] = new LinkedList<>();
```

Two Sum

- Given an array of integers nums and an integer target, return indices of the two numbers such that they add up to target.
- You may assume that each input would have exactly one solution, and you may not use the same element twice.
- You can return the answer in any order.

- Example 1:
  - Input: nums = [2,7,11,15], target = 9
  - Output: [0,1]

- Example 2:
  - Input: nums = [3,2,4], target = 6
  - Output: [1,2]

- Example 3:
  - Input: nums = [3,3], target = 6
  - Output: [0,1]

```
int[] twoSum (int nums[], int target) {
    for( i= 0 ; i< nums. length-1  ; i++) {
        for( j= i+1; j < nums. length  ; j++) {
            if(nums[i] + nums[j] == target)
                return new int[ ]{ i , j} ;
        }
    }
    return null;
}
```

n times → for loop

n times → for loop

$T(n) = O(n^2)$

$S(n) = O(1)$

Two Sum

- Given an array of integers nums and an integer target, return indices of the two numbers such that they add up to target.
- You may assume that each input would have exactly one solution, and you may not use the same element twice.
- You can return the answer in any order.

diff = target - num[i]

- Example 1:
  - Input: nums = [2,7,11,15], target = 9
  - Output: [0,1]

9-2=7
9-7=2

key	value
2	0

T(n) = O(n)
S(n) = O(n)

- Example 2:
  - Input: nums = [3,2,4], target = 6
  - Output: [1,2]

6-3=3
6-2=4
6-4=2

key	value
3	0
2	1

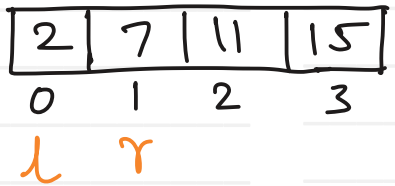
- Example 3:
  - Input: nums = [3,3], target = 6
  - Output: [0,1]

- two pointers or index variables are used to traverse the collection
- pointers can be moved
    1. in same direction
    2. in opposite direction
    3. with different speed
    4. over two different collections

Two Sum II - Input Array Is Sorted

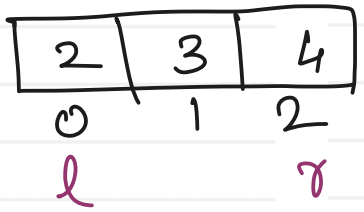
- Given a 1-indexed array of integers numbers that is already sorted in non-decreasing order, find two numbers such that they add up to a specific target number. Let these two numbers be numbers[index1] and numbers[index2] where 1 <= index1 < index2 <= numbers.length.
- Return the indices of the two numbers index1 and index2, each incremented by one, as an integer array [index1, index2] of length 2.
- The tests are generated such that there is exactly one solution. You may not use the same element twice.
- Your solution must use only constant extra space.

- Example 1:
- Input: numbers = [2,7,11,15], target = 9
- Output: [1,2]



l	r	nums[l] + nums[r]	target=9
0	3	17	>
0	2	13	>
0	1	9	=

- Example 2:
- Input: numbers = [2,3,4], target = 6
- Output: [1,3]

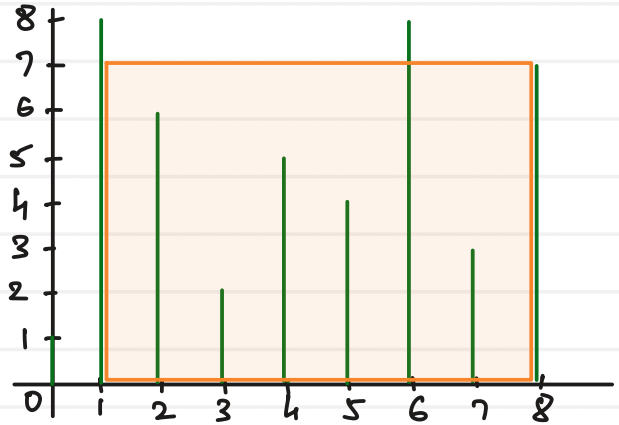


l	r	nums[l] + nums[r]	target=6
0	2	6	=

- Example 3:
- Input: numbers = [-1,0], target = -1
- Output: [1,2]

Container With Most Water

- You are given an integer array height of length n. There are n vertical lines drawn such that the two endpoints of the ith line are (i, 0) and (i, height[i]).
- Find two lines that together with the x-axis form a container, such that the container contains the most water.
- Return the maximum amount of water a container can store.
- Notice that you may not slant the container.



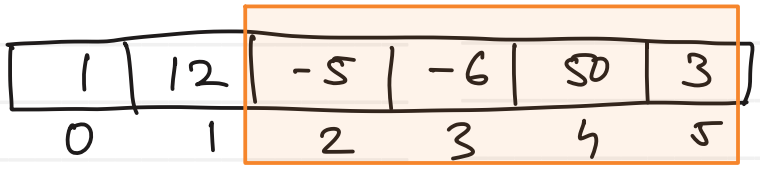
- Example 1:
  - Input: height = [1,8,6,2,5,4,8,3,7]
  - Output: 49
  - Explanation: The above vertical lines are represented by array [1,8,6,2,5,4,8,3,7]. In this case, the max area of water (blue section) the container can contain is 49.

- Example 2:
  - Input: height = [1,1]
  - Output: 1

left	right	height	width	area	maxArea
0	8	1	8	8	0 8 49
1	8	7	7	49	
1	7	3	6	18	
1	6	8	5	40	
2	6	6	4	24	
3	6	2	3	6	

Maximum Average Subarray I

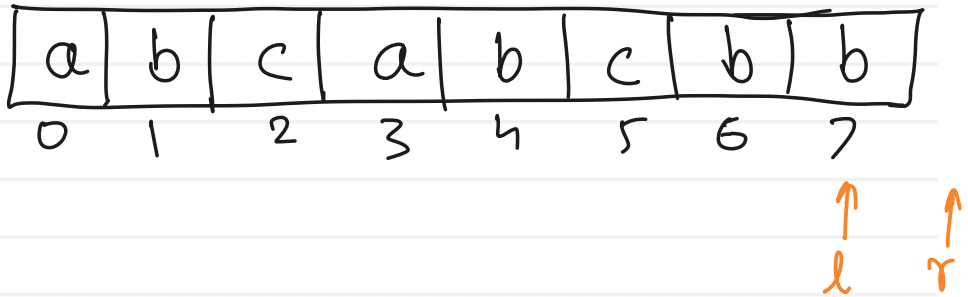
- You are given an integer array nums consisting of n elements, and an integer k.
- Find a contiguous subarray whose length is equal to k that has the maximum average value and return this value. Any answer with a calculation error less than 10-5 will be accepted.
- Example 1:
  - Input: nums = [1,12,-5,-6,50,3], k = 4
  - Output: 12.75000
  - Explanation: Maximum average is (12 - 5 - 6 + 50) / 4 = 51 / 4 = 12.75
- Example 2:
  - Input: nums = [5], k = 1
  - Output: 5.00000



① 1+12-5-6 = 2
② 12-5-6+50 = 51
③ -5-6+50+3 = 42

Longest Substring Without Repeating Characters

- Given a string s, find the length of the longest substring without duplicate characters.



- Example 1:
  - Input: s = "abcabcbb"
  - Output: 3
  - Explanation: The answer is "abc", with the length of 3. Note that "bca" and "cab" are also correct answers.

l r
0 0
1
2
3 X
1 ✓
4 X
2

maxLength
4
3
3

- Example 2:
  - Input: s = "bbbbb"
  - Output: 1
  - Explanation: The answer is "b", with the length of 1.

- Example 3:
  - Input: s = "pwwkew"
  - Output: 3
  - Explanation: The answer is "wke", with the length of 3.
  - Notice that the answer must be a substring, "pwke" is a subsequence and not a substring.